DUNGEON 1: THE WEB

COMPUTER SCIENCE DUNGEON ADVENTURES

pencil · paper · one die

You are a brand-new **adventurer**: a bot that works like the **web crawlers** search engines send out to read the internet. Your first job is an old website, five floors deep, that shuts down forever on Friday. Explore each floor, index its **DATA**, and survive the broken code that drains your **RAM**. Reach the RUN square to follow the link down to the next floor. One warning from the last adventurer sent here: something at the bottom has been sorting visitors all wrong.

DRAW YOUR ADVENTURER

ADVENTURER NAME _____

REAL NAME _____

CLASS _____

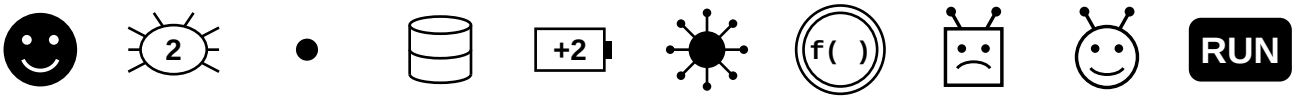
DATE STARTED _____

DATE FINISHED _____

YOU START WITH

10	0	10
RAM	**DATA**	**MAX RAM**

RAM is your energy. Broken code drains it.
DATA is what you index. Spend it in the shop.
MAX RAM is the most RAM you can hold.



Part of the Computer Science (CS) Offline project

© 2026 Mike Renne · CC BY-NC-SA 4.0 · free to copy for classrooms

HOW TO PLAY

Goal: get through all 5 floors. Collect DATA. Do not run out of RAM. **You need:** this booklet, a pencil and one die.

YOUR TURN

1. Roll the die.

2. Odd or even?
ODD (1, 3, 5): move **diagonally**.
EVEN (2, 4, 6): move **straight** (up, down, left or right).

3. Pick a direction and move that many squares, one square at a time.

4. Draw your path as you go. Mark the square where you stop with an **X**. Start there next turn.

5. Use everything you cross. Every bug, data dot and power-up on your path happens to you, in order. Write each gain or loss in the boxes at the bottom of the page, then cross the thing out. It only works once.

MOVING RULES

• If any direction lets you make your **whole move without hitting a wall**, you must pick one of those directions.

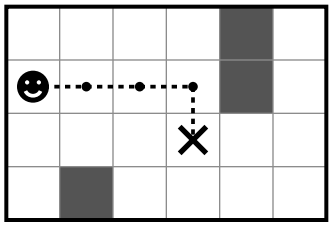
• **Keep going the same way.** You turn only when a wall or the edge of the map blocks you. Then pick a new direction of the same kind and keep counting.

• You may **not** turn to dodge a bug or to grab something.

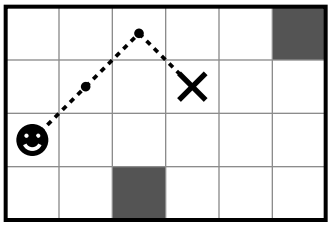
• Go back the way you came **only** if there is no other way.

• Moving diagonally, only the square you land on matters. You can slip between two wall corners.

EXAMPLES



Rolled 4 (even). Moved right 3 squares. A wall blocked the way, so turned and finished the last square going down.



Rolled 3 (odd). Moved up-right 2 squares. The edge blocked the way, so bounced down-right for the last square.

LEAVING A FLOOR

Stop on the **RUN** square, stop on any square touching it (corners count), or pass over it. Then you may leave. You can also stay to collect more.

Now do the math, once for RAM and once for DATA:
START + gained – lost = END

Copy both END numbers to START on the next floor.

RAM cannot go above your Max RAM. DATA cannot go below 0. You do not crash in the middle of a floor: only the END number counts.

CRASH!

If your END RAM is **0 or less**, your adventurer crashed. Tally one crash on the last page. Start the next floor with **10 RAM and 0 DATA**. Keep playing!

CODE REVIEW

Real coders check each other's work. After every floor, swap booklets with a partner. Check their path, their math and their floor puzzle. Found a mistake? Fix it together.

© 2026 Mike Renne · CC BY-NC-SA 4.0 · free to copy for classrooms

FLOOR 1 THE HOME PAGE

Algorithms: follow steps in order, repeat with loops

FLOOR RULE: PROGRAM SQUARES

A program is a list of steps. Step on a program square (**P1**, **P2**, **P3**) and your move stops: the rest of your roll is lost. Now **run the program**: move one square for each arrow, in order.

REPEAT 2 [...] is a loop. Do the steps inside the brackets 2 times.

Use everything you cross. You stop where the program ends. Each program runs once, then cross it out.

THE PROGRAMS

P1 → → ↑ ↑

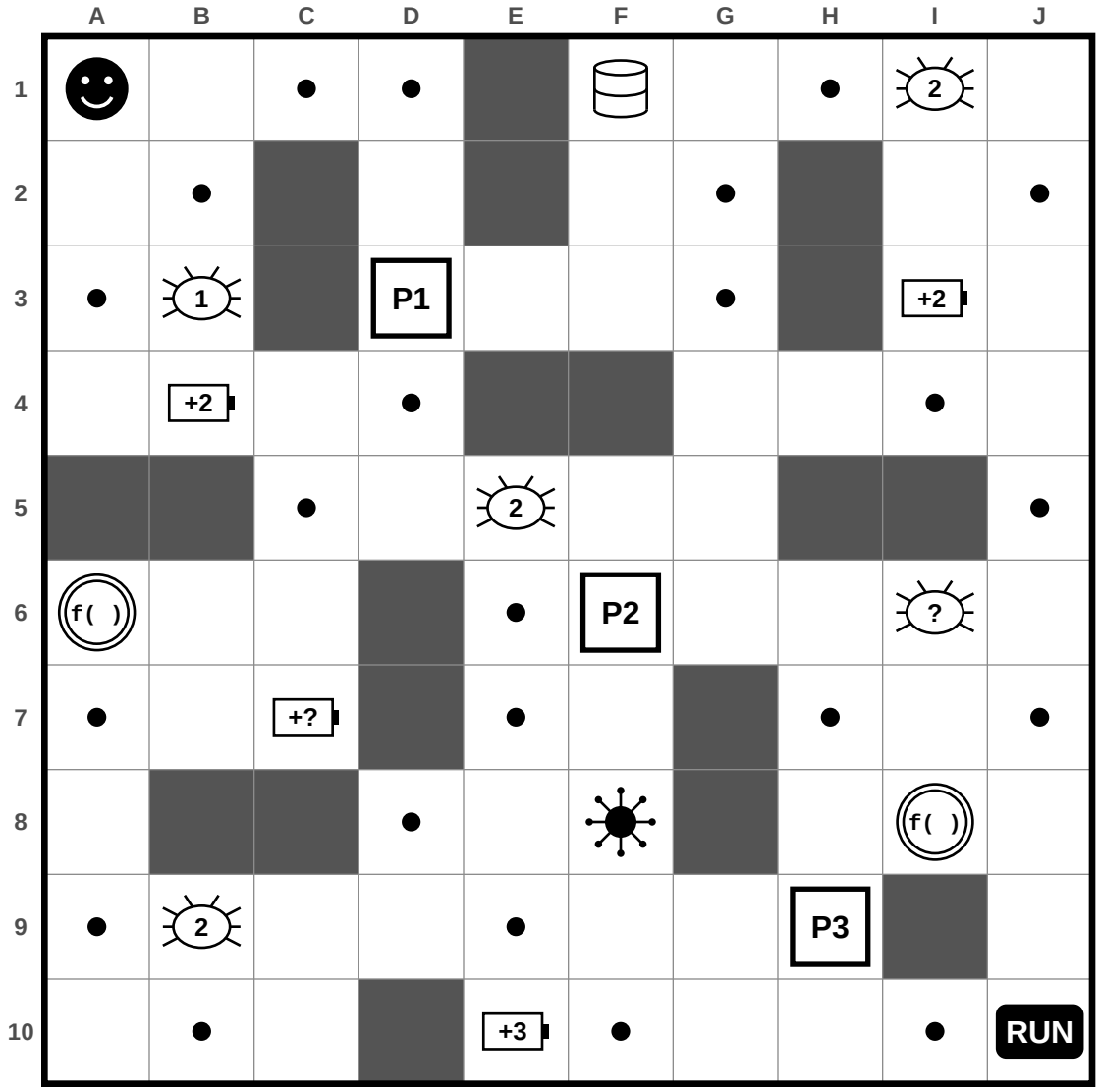
I ended on square: _____

P2 REPEAT 2 [↓ ←]

I ended on square: _____

P3 ↓ REPEAT 2 [→]

I ended on square: _____



KEY

- **Start**
- **Exit.** Stop on it, next to it, or pass over it
- **Data:** +1 DATA
- **Database:** roll, gain that much DATA
- **Bug:** lose that much RAM
- **Mystery bug:** roll, lose that much RAM
- **Power-up:** gain that much RAM
- **Mystery power-up:** roll, gain that RAM
- **Malware:** STOP, even in a program. Roll, lose that DATA
- **Function:** jump to the other f( ) every time, keep going
- **Program:** stop and run it

	START	+ GAINED	– LOST	END
RAM				
DATA				

END = START + gained – lost. RAM cannot go above your Max RAM. DATA cannot go below 0. END RAM of 0 or less = CRASH.

© 2026 Mike Renne · CC BY-NC-SA 4.0 · free to copy for classrooms

FLOOR 2 THE SOURCE CODE

Programming: variables and IF / ELSE

FLOOR RULE: THE VARIABLE x

A **variable** is a box that holds a value. Yours is called **x** and it starts at **1**.

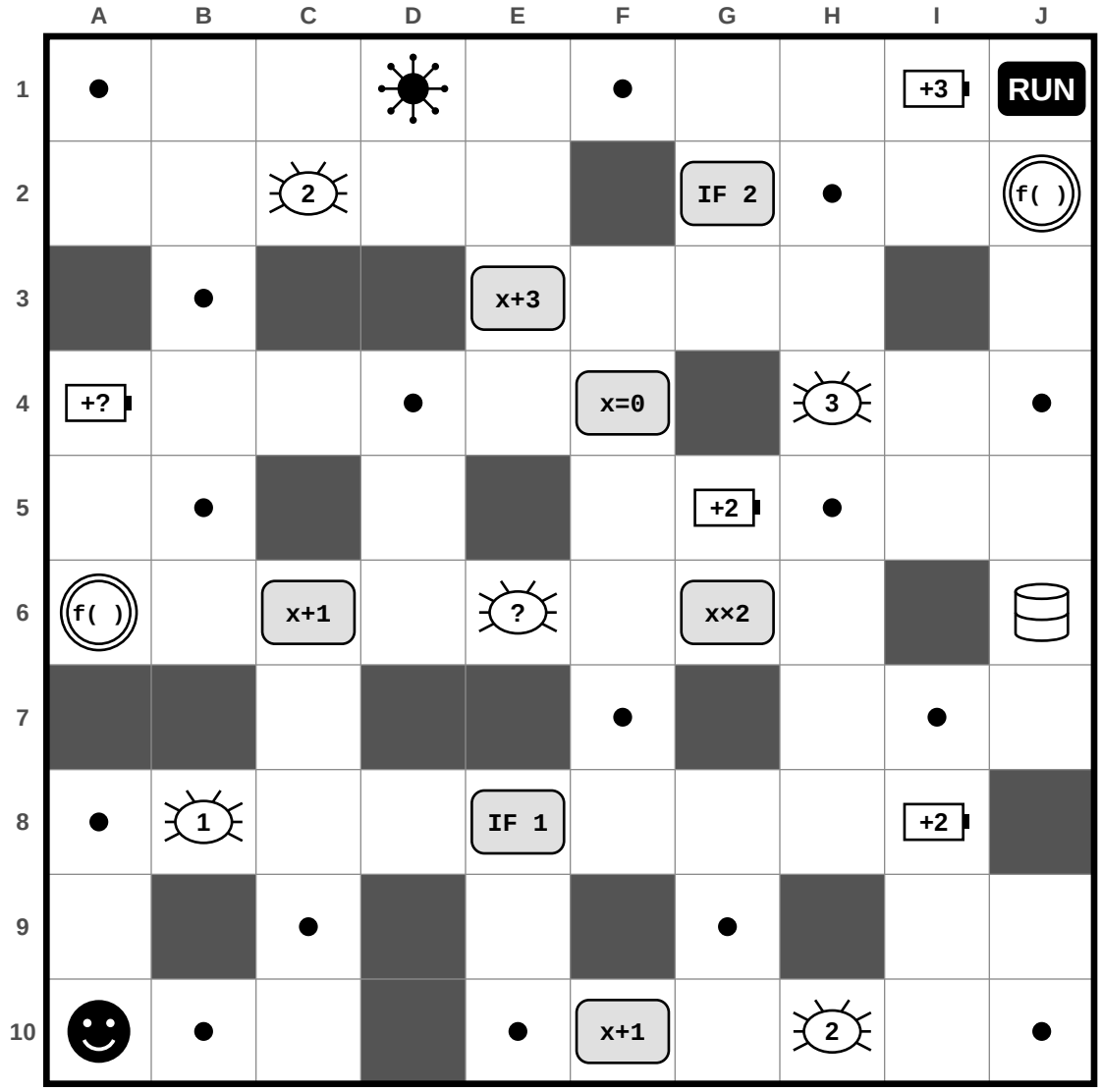
Cross a code square: change x, cross out the old value and write the new one in the next box. Careful: **x=0** is a bug that wipes your variable!

When you leave the floor: **gain x DATA**.

X = | 1 | | | | | | |

IF 1
```
IF x > 3:      +3 RAM
ELSE:          −2 RAM
```

IF 2
```
IF x is even:  +4 DATA
ELSE:          −1 RAM
```



KEY

- **Start**
- **Exit.** Stop on it, next to it, or pass over it
- **Data:** +1 DATA
- **Database:** roll, gain that much DATA
- **Bug:** lose that much RAM
- **Mystery bug:** roll, lose that much RAM
- **Power-up:** gain that much RAM
- **Mystery power-up:** roll, gain that RAM
- **Malware:** STOP, even in a program. Roll, lose that DATA
- **Function:** jump to the other f( ) every time, keep going
- **Code:** change x
- **IF:** check x, do what it says

	START	+ GAINED	− LOST	END
RAM				
DATA				

END = START + gained − lost. RAM cannot go above your Max RAM. DATA cannot go below 0. END RAM of 0 or less = CRASH.

© 2026 Mike Renne · CC BY-NC-SA 4.0 · free to copy for classrooms

FLOOR 3 THE DATABASE

Data: binary numbers, collecting and reading data

FLOOR RULE: EVERYTHING IS BINARY

Computers store every number using only **1** and **0**. On this floor, bug strength and chest values are in **binary**.

Each place is worth 8, 4, 2 or 1. Add up the places that hold a 1.

A door shows a binary number. It opens only if you hold the key with the same number.

EXAMPLE

8	4	2	1	
0	1	0	1	= 4 + 1 = 5

MY CONVERSIONS (binary = number)

____ = ___ ____ = ___ ____ = ___

____ = ___ ____ = ___ ____ = ___

____ = ___ ____ = ___ ____ = ___

ROLL LOG

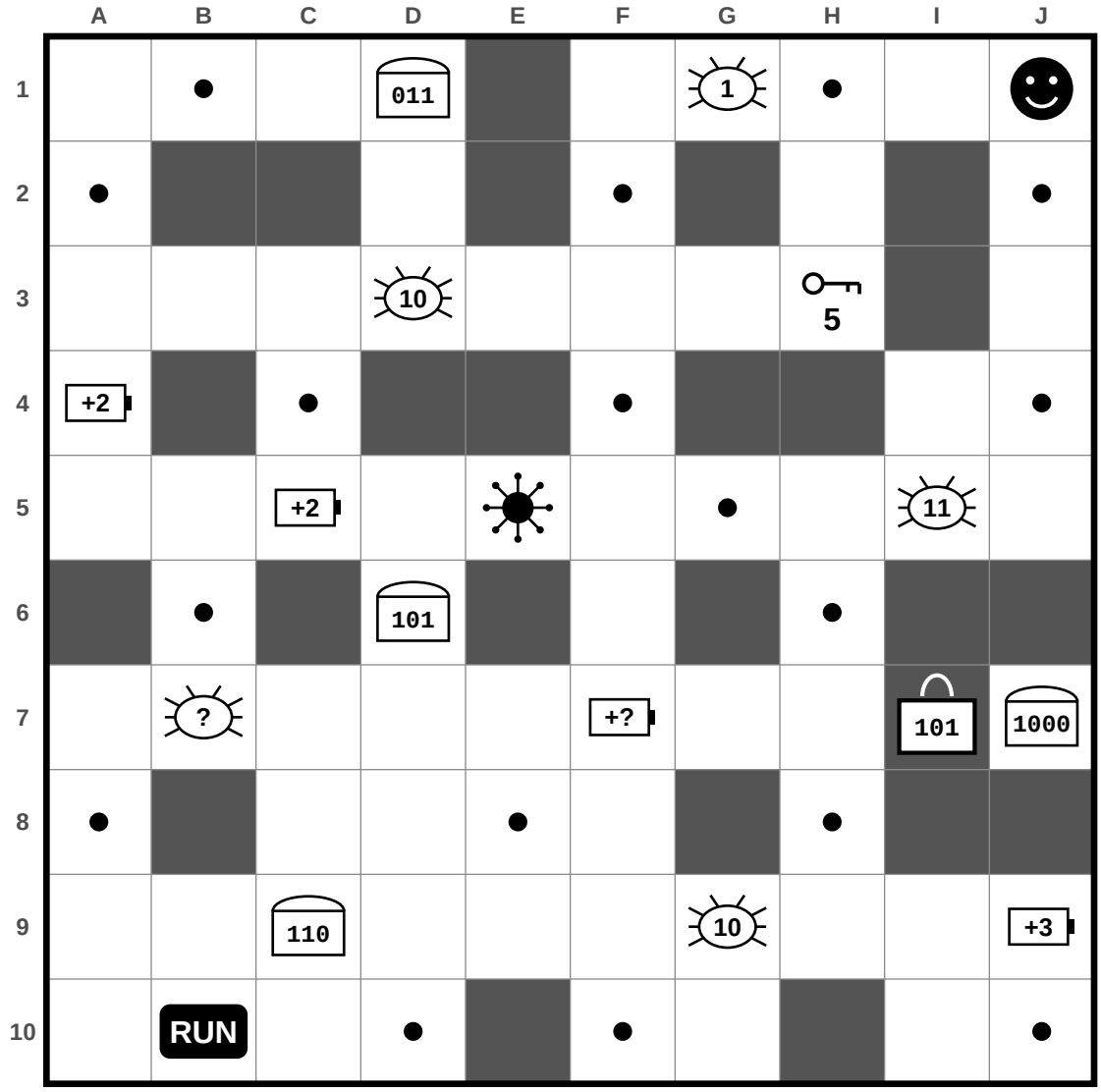
Tally every roll you make.

1	
2	
3	
4	
5	
6	

Most common roll: ____

Total rolls: ____

Log complete: +2 DATA



KEY

- **Start**
- **Exit.** Stop on it, next to it, or pass over it
- **Data:** +1 DATA
- **Database:** roll, gain that much DATA
- **Bug:** lose that much RAM
- **Mystery bug:** roll, lose that much RAM
- **Power-up:** gain that much RAM
- **Mystery power-up:** roll, gain that RAM
- **Malware:** STOP, even in a program. Roll, lose that DATA
- **Bit chest:** gain DATA (binary)
- **Key** for a door
- **Door:** a wall until you hold its key

	START	+ GAINED	– LOST	END
RAM				
DATA				

END = START + gained – lost. RAM cannot go above your Max RAM. DATA cannot go below 0. END RAM of 0 or less = CRASH.

© 2026 Mike Renne · CC BY-NC-SA 4.0 · free to copy for classrooms

FLOOR 4 THE LOGIN PAGE

Security: passwords, keys and secret codes

FLOOR RULE: SECRET CODES

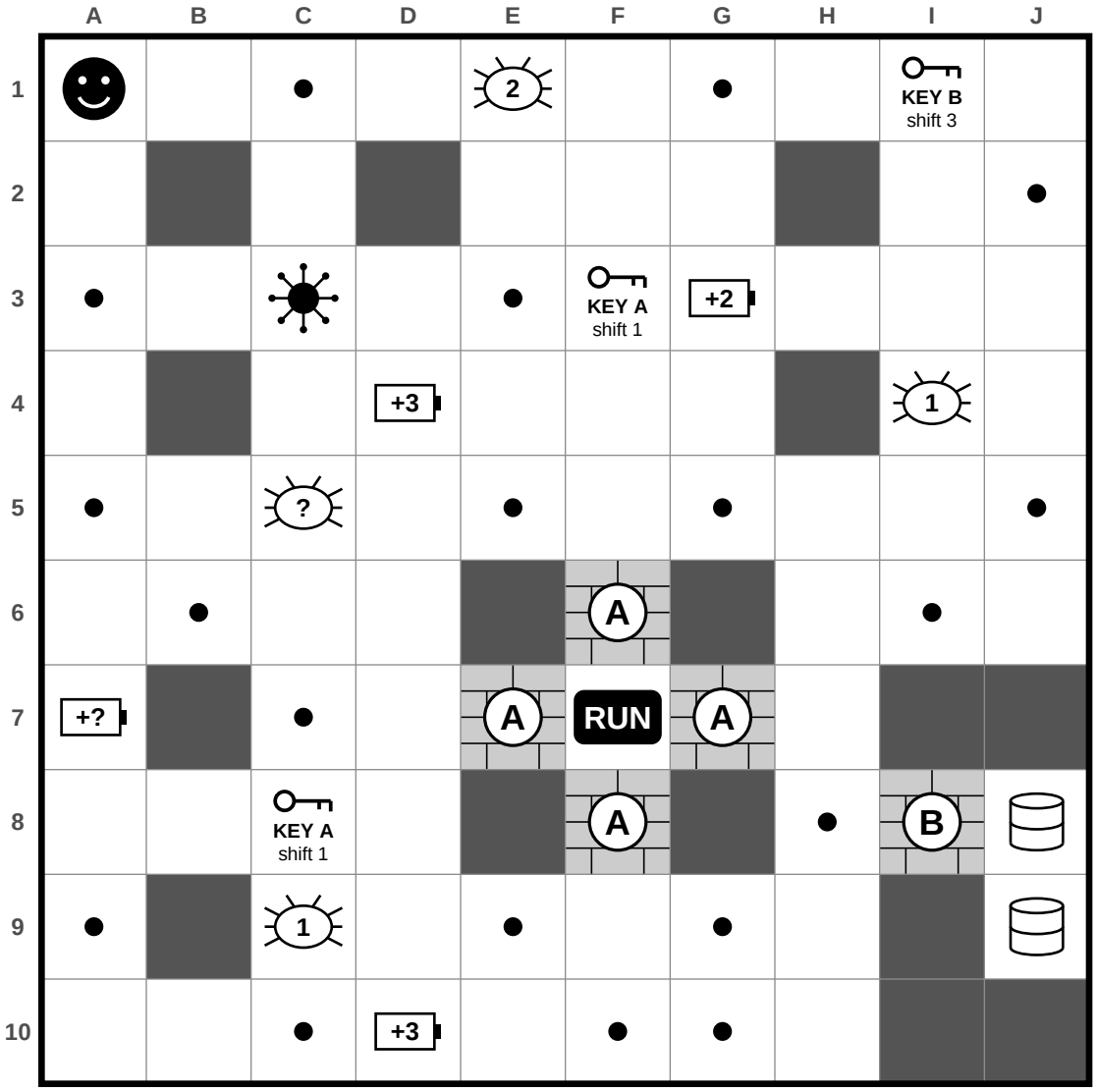
Each firewall is locked with a scrambled password. A **key** tells you how many letters to shift **back** along the alphabet to unscramble it. Example: with shift 2, **D C V** becomes **B A T**.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

Firewall A: **C Z U F** shift: ____ password: ___ ___ ___ ___

Firewall B: **F R G H** shift: ____ password: ___ ___ ___ ___

Holding the key? Decode the password, write it here, and walk through free. **No key?** You can smash through, but it costs **2 RAM**. Wrong password at code review: –3 DATA.



KEY

- **Start**
- **Exit.** Stop on it, next to it, or pass over it
- **Data:** +1 DATA
- **Database:** roll, gain that much DATA
- **Bug:** lose that much RAM
- **Mystery bug:** roll, lose that much RAM
- **Power-up:** gain that much RAM
- **Mystery power-up:** roll, gain that RAM
- **Malware:** STOP, even in a program. Roll, lose that DATA
- **Cipher key**
- **Firewall:** free with key + password. No key: –2 RAM

	START	+ GAINED	– LOST	END
RAM				
DATA				

END = START + gained – lost. RAM cannot go above your Max RAM. DATA cannot go below 0. END RAM of 0 or less = CRASH.

© 2026 Mike Renne · CC BY-NC-SA 4.0 · free to copy for classrooms

FLOOR 5 THE SORTING ROOM

AI: how the site's old AI sorts things, and when it gets it wrong

FLOOR RULE: THE SORTING AI

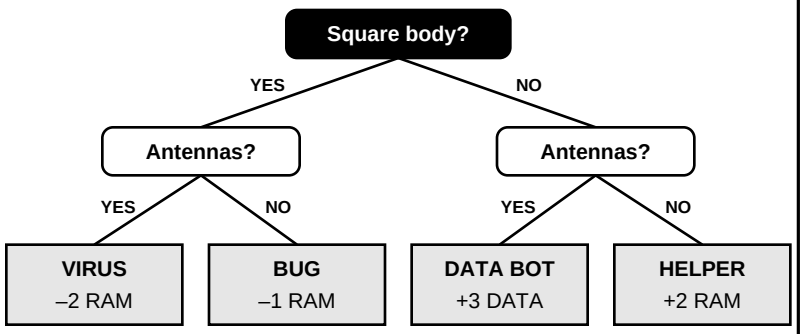
You found it: the site's old **sorting AI**, and you cannot overrule it. For every bot you meet, follow its tree and do what it says, **even if it seems wrong**.

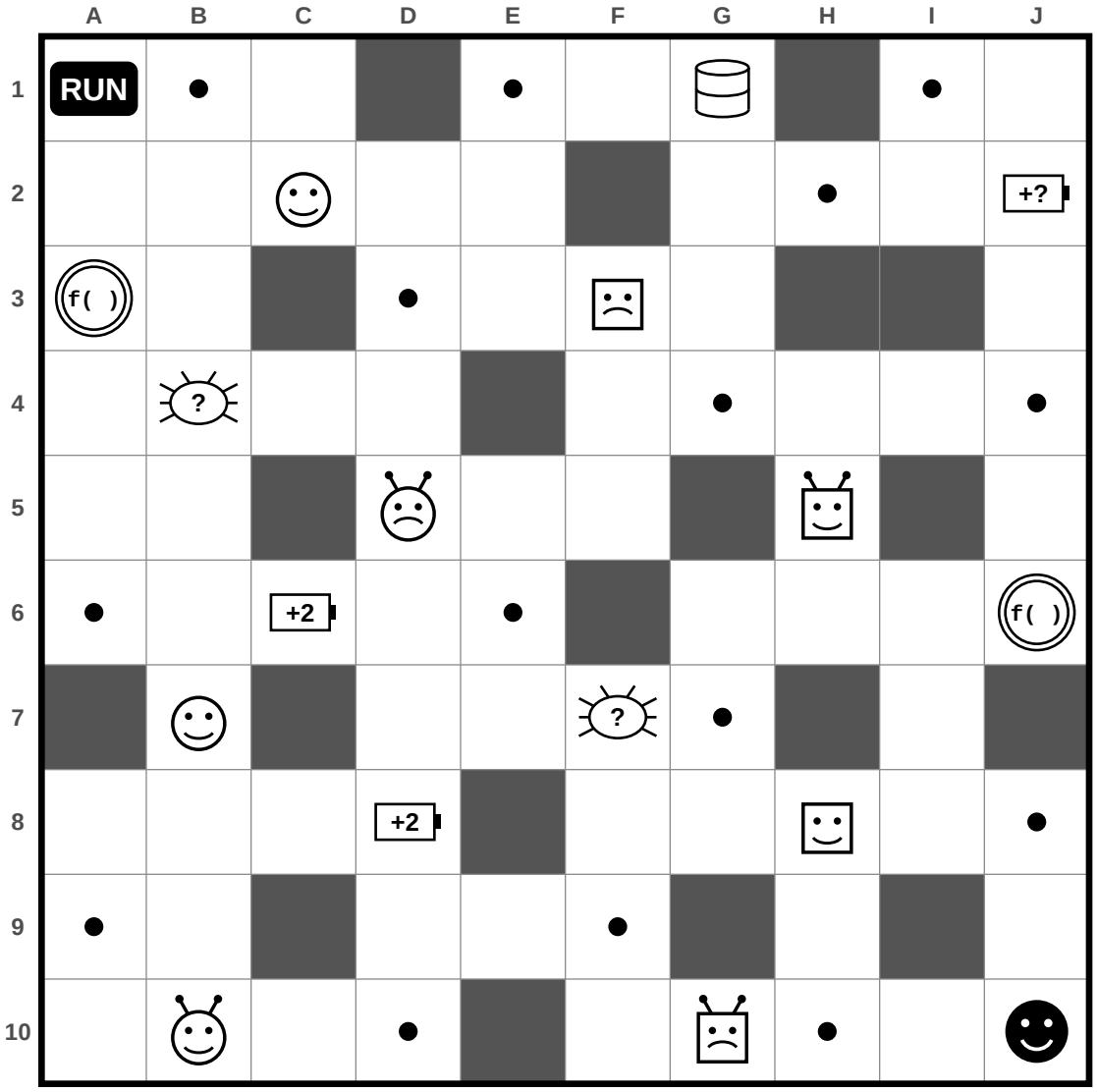
The truth: **smiling** bots are friendly, **frowning** bots are hostile. Its builders trained it on body shape and antennas only. It never learned to look at faces.

Friendly bots the AI hurt: ____

Hostile bots the AI trusted: ____

A better question for the tree: _____





KEY

- **Start**
- **Exit.** Stop on it, next to it, or pass over it
- **Data:** +1 DATA
- **Database:** roll, gain that much DATA
- **Bug:** lose that much RAM
- **Mystery bug:** roll, lose that much RAM
- **Power-up:** gain that much RAM
- **Mystery power-up:** roll, gain that RAM
- **Malware:** STOP, even in a program. Roll, lose that DATA
- **Function:** jump to the other f( ) every time, keep going
- **Bot:** sort it with the AI's tree

	START	+ GAINED	– LOST	END
RAM				
DATA				

END = START + gained – lost. RAM cannot go above your Max RAM. DATA cannot go below 0. END RAM of 0 or less = CRASH.

© 2026 Mike Renne · CC BY-NC-SA 4.0 · free to copy for classrooms

THE SHOP

Visit the shop twice: **after Floor 2** and **after Floor 4**. Pay with DATA. Write the price in the **– LOST** box for DATA on the next floor. You can buy each thing once per visit.

ITEM	PRICE	WHAT IT DOES	SHOP 1	SHOP 2
PATCH	4 DATA	Gain 4 RAM. Write +4 in the + GAINED box for RAM on the next floor.	☐ bought	☐ bought
DEBUGGER	3 DATA	Roll again once, on any roll you do not like.	☐ bought ☐ used	☐ bought ☐ used
ANTIVIRUS	5 DATA	Ignore one bug or one malware. Cross it out without losing anything.	☐ bought ☐ used	☐ bought ☐ used
RAM UPGRADE	8 DATA	Your Max RAM goes up by 2 for the rest of the game.	☐ bought	☐ bought

MY MAX RAM: **10** → ☐ → ☐

FINAL STATS

CRASHES (tally)	FINAL RAM	FINAL DATA	FLOORS CLEARED

YOUR RANK (by final DATA)

- ☐ **Intern** — 0 – 14 DATA
- ☐ **Junior Coder** — 15 – 29 DATA
- ☐ **Senior Engineer** — 30 – 44 DATA
- ☐ **System Architect** — 45 + DATA

THINK BACK

Which floor puzzle was hardest? What made it hard?

On Floor 5, was the sorting AI fair to every bot? Why or why not?

Where have you seen one of these ideas in real life?

© 2026 Mike Renne · CC BY-NC-SA 4.0 · free to copy for classrooms

TEACHER PAGE: ANSWER KEY (do not hand out)

FLOOR 1: programs
P1 ends on **F1**
P2 ends on **D8**
P3 ends on **J10**

FLOOR 2: variable x
x starts at 1. Order matters: x+1 then x×2 gives 4, but x×2 then x+1 gives 3. The x=0 square models a bug that overwrites a variable. Players gain x DATA on leaving.

FLOOR 3: binary
1 = 1, 10 = 2, 11 = 3, 011 = 3, 101 = 5, 110 = 6, 1000 = 8.
The door opens with key 5. Roll log: any complete tally earns +2 DATA.

FLOOR 4: cipher
Firewall A: CZUF, shift 1 = **BYTE**
Firewall B: FRGH, shift 3 = **CODE**

FLOOR 5: bots
C2: round, no antennas, smiling = **HELPER**
F3: square, no antennas, frowning = **BUG**
D5: round, antennas, frowning = **DATA BOT** (sorting AI is wrong)
H5: square, antennas, smiling = **VIRUS** (sorting AI is wrong)
B7: round, no antennas, smiling = **HELPER**
H8: square, no antennas, smiling = **BUG** (sorting AI is wrong)
B10: round, antennas, smiling = **DATA BOT**
G10: square, antennas, frowning = **VIRUS**

WHAT EACH FLOOR TEACHES
1. Algorithms and Design: sequence, loops, tracing a program.
2. Programming: variables, assignment, IF / ELSE.
3. Data and Analysis: binary representation, collecting and summarising data (the roll log).
4. Systems and Security: authentication, encryption, the cost of weak defences.
5. Computing and Society (AI): classification with a decision tree, and bias from features a model was never trained on.
Match these to your state or CSTA standard codes before use; the codes are not listed here.

RUNNING IT
• Each student needs a booklet, a pencil and a die. Print single-sided so nothing shows through the maps.
• Teach the movement rules with Floor 1 on a projector or the board, playing 3 or 4 turns together.
• In computer playtests a floor took about 9 to 11 turns, and 9 in 10 finished within 22. Real classroom timing is not yet tested: time your first session.
• About 4 in 10 simulated players crashed at least once. A crash is a reset, not a loss.
• Code review pairs replace teacher marking. Keep this page for disputes.

DISCUSSION STARTERS
1. A program square moved you without a die. What makes a program predictable?
2. Did the order you crossed the code squares change x? Why?
3. Was your most common roll the same as your partner's? What would a class-wide log show?
4. Smashing a firewall cost RAM. What does a break-in cost a real organisation?
5. The sorting AI hurt friendly square bots. Who is responsible when a model is unfair: the model, its training data, or the people who built it?

© 2026 Mike Renne · CC BY-NC-SA 4.0 · free to copy for classrooms